

Хранение данных в Веб

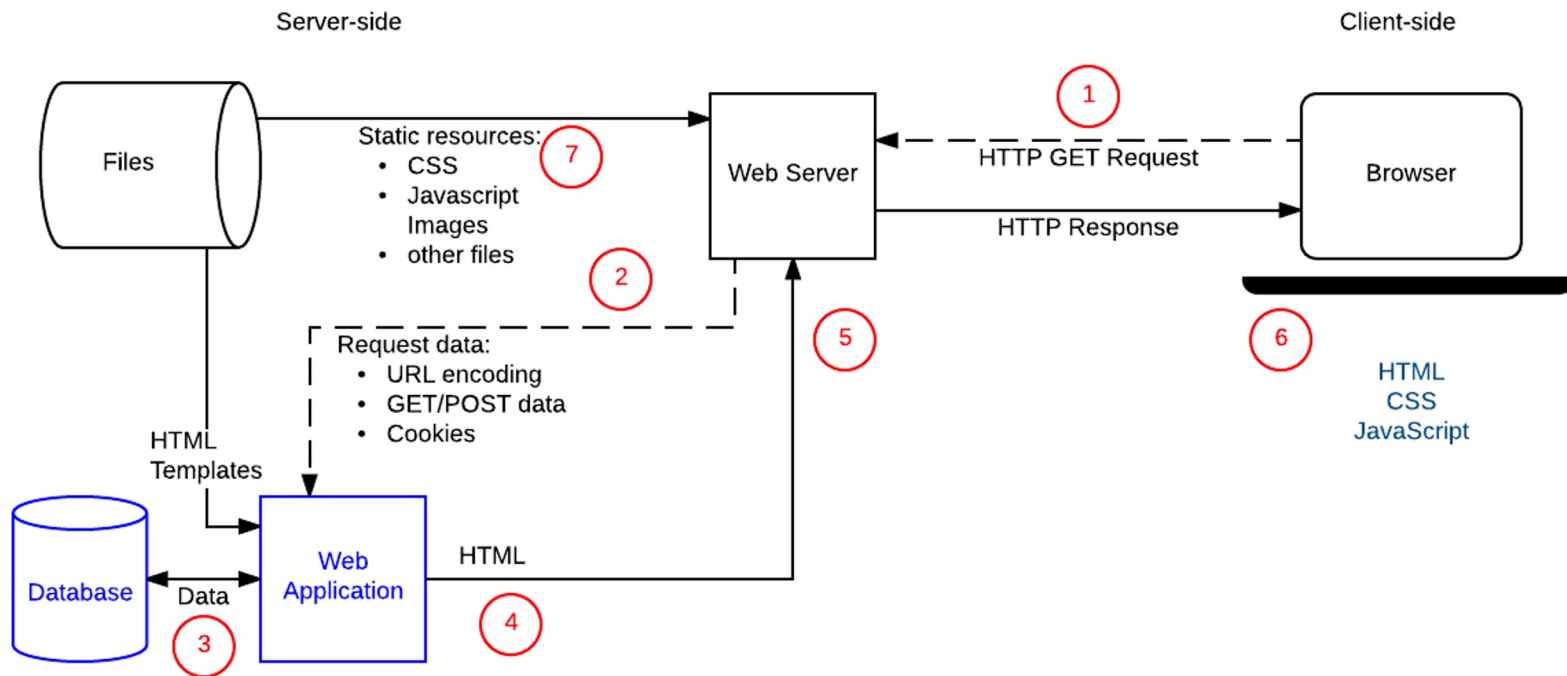
Александр Менщиков,
к.т.н., доцент ИТМО

Содержание лекции

1. Хранение информации в браузере
2. Фингерпринтинг
3. Редиректы
4. Основы аутентификации
5. AJAX
6. API и REST
7. Форматы данных
8. Прокси

Хранение информации в браузере

Анатомия запроса в Веб



Зачем хранить информацию на стороне клиента

- Персонализация настроек сайта (например, отображение выбранных пользователем виджетов, цветовой схемы или размера шрифта)
- Сохранение предыдущих действий на сайте (например, хранение содержимого корзины предыдущего сеанса, запоминание того, был ли пользователь ранее зарегистрирован на сайте).
- Сохранение данных, чтобы сайт можно было быстрее (и потенциально дешевле) загрузить или использовать без подключения к сети.

Основные технологии сохранения информации в браузере

- Cookie
- Web Storage API
- IndexedDB API
- Service workers и Cache API*

** рассмотрим позднее*

Зашифрованные Cookie

eyJzZXNzaW9uIjoiaZlKMWMYVnlibUZ0WlNjNk1DSmhaRzFwYmlJc0lDSndZW
E56ZDI5eVpDSTZJQ0p4ZDJWeWRIa2lmUT09In0.ZeHYPg.aiA93oXa_4Ua6cy
_YGPRkjAp63I

<https://flask.palletsprojects.com/en/2.3.x/quickstart/#sessions>



```
resp = make_response(f"Current cookie is {request.cookies}")
resp.set_cookie('session',
base64.b64encode(json.dumps({"username": "admin", "password":
"qwerty"}).encode()).decode())
return resp
```



```
session['session'] = base64.b64encode(json.dumps({"username":
"admin", "password": "qwerty"}).encode()).decode()
return f"Current cookie is {request.cookies}"
```

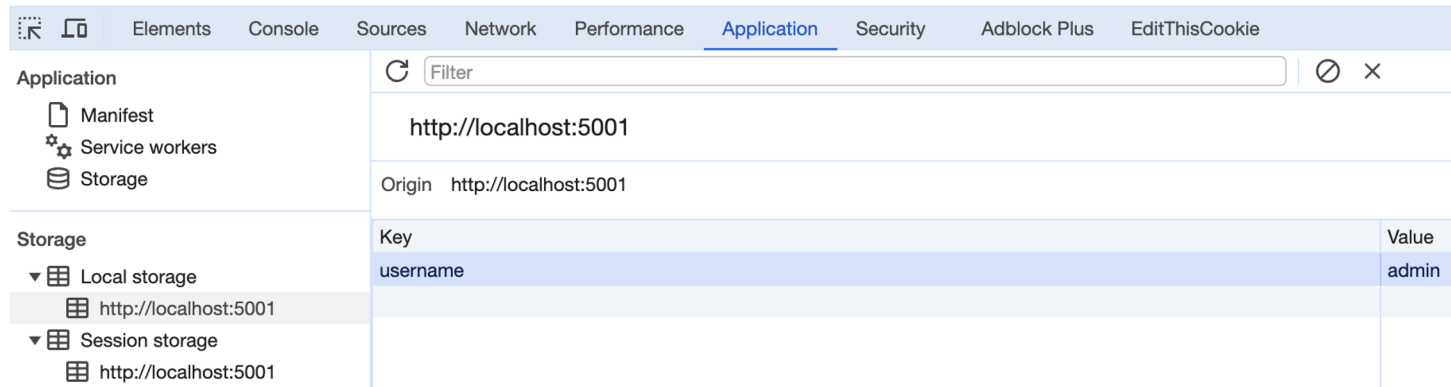
<https://github.com/noraj/flask-session-cookie-manager>

Web Storage API

- Сессионное хранилище (sessionStorage) обслуживает область хранения данных для каждого домена, доступное на протяжении сессии (пока браузер открыт, даже в случае перезагрузки страницы)
- Локальное хранилище (localStorage) делает то же самое, но сохраняет данные навечно

Особенности:

- key и value должны быть строками. Лимит 5 Мб+, зависит от браузера. Данные не имеют «времени истечения». Данные привязаны к источнику (домен/протокол/порт).



The screenshot shows the 'Application' tab in a web browser's developer tools. The left sidebar is expanded to 'Storage' > 'Local storage'. It shows a single entry for 'http://localhost:5001'. The right pane displays the details for this entry, showing the 'Origin' as 'http://localhost:5001' and a table of stored data.

Key	Value
username	admin

Web Storage API – JS

window.localStorage

```
> window.localStorage
< ▶ Storage {username: 'admin', length: 1}
> window.localStorage['username'] = '123'
< '123'
> window.localStorage
< ▶ Storage {username: '123', length: 1}
> window.localStorage['username']
< '123'
```

window.sessionStorage

```
> window.sessionStorage
< ▶ Storage {length: 0}
> window.sessionStorage['secret'] = 'qwe'
< 'qwe'
> window.sessionStorage
< ▶ Storage {secret: 'qwe', length: 1}
> window.sessionStorage['secret']
< 'qwe'
```

Можно подписаться на изменения

<https://mdn.github.io/dom-examples/web-storage/event.html>

<https://html.spec.whatwg.org/multipage/webstorage.html#webstorage>

В «Инкогнито» работает
по-разному

<https://mdn.github.io>

Origin <https://mdn.github.io>

Key	Value
bgcolor	EDFF4A
image	images/firefoxos.png
font	monospace

```
window.addEventListener('storage', function(e) {  
  document.querySelector('.my-key').textContent = e.key;  
  document.querySelector('.my-old').textContent = e.oldValue;  
  document.querySelector('.my-new').textContent = e.newValue;  
  document.querySelector('.my-url').textContent = e.url;  
  document.querySelector('.my-storage').textContent =  
    JSON.stringify(e.storageArea);  
});
```

IndexedDB API

Низкоуровневое API для клиентского хранилища большого объёма структурированных данных (в отличии от Web Storage)



The screenshot shows the Application tab in Chrome DevTools. The left sidebar has a tree view with 'Application' expanded, showing 'Storage' > 'IndexedDB' > 'MyDatabase' > 'MyObjectStore'. The main panel displays a table of IndexedDB entries. The table has four columns: '#', 'Key (Key path: ["name.last", "name.first"])', 'Primary key (Key path: "id")', and 'Value'. There are two entries. Entry 0 has a key of ['Doe', 'John'] and a primary key of 12345. Entry 1 has a key of ['Smith', 'Bob'] and a primary key of 67890. The values are JSON objects containing id, name (with first and last properties), and age.

#	Key (Key path: ["name.last", "name.first"])	Primary key (Key path: "id")	Value
0	(2) ['Doe', 'John'] 0: "Doe" 1: "John" - length: 2	12345	{id: 12345, name: {...}, age: 42} - age: 42 - id: 12345 - name: - first: "John" - last: "Doe"
1	(2) ['Smith', 'Bob'] 0: "Smith" 1: "Bob" - length: 2	67890	{id: 67890, name: {...}, age: 35} - age: 35 - id: 67890 - name: - first: "Bob" - last: "Smith"

Изучаем IndexedDB API

https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API/Using_IndexedDB

[https://developer.mozilla.org/en-](https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria#how_much_data_can_be_stored)

[US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria#how_much_data_can_be_stored](https://developer.mozilla.org/en-US/docs/Web/API/Storage_API/Storage_quotas_and_eviction_criteria#how_much_data_can_be_stored)

10% диска или 10 GiB

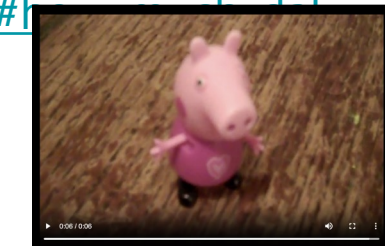


```
const request = window.indexedDB.open("videos_db");
const db = request.result;
const transaction = db.transaction(["videos_os"], "readwrite");
const objectStore = transaction.objectStore("videos_os");
const objectStoreRequest = objectStore.get("pig");
let videoUrl = URL.createObjectURL(objectStoreRequest.result.mp4);

console.log(videoUrl);
// blob:https://mdn.github.io/c4995d8e-d6d2-433d-8bf5-c359b6aeb2b7
```



#	Key (Key path: "name")
0	"crystal"
1	"elf"
2	"frog"
3	"monster"
4	"pig"
5	"rabbit"



- IndexedDB
 - toDoList
 - videos_db
 - videos_os
 - mp4
 - webm

Фингерпринтинг

Методы фингерпринтинга

- Hardware benchmarking
- Использование Canvas and WebGL (детектим шрифты и GPU)
- Анализ активных браузерных приложений

`> navigator.plugins`

- История браузера

`CSS selector :visited`

- Трюки с favicon

FingerprintJS

<https://fingerprintjs.github.io/fingerprintjs/>

<https://github.com/fingerprintjs/fingerprintjs>

Visitor identifier:

af92d304af8100a881d448a7d622d2e4

Time took to get the identifier:

99ms

Confidence score:

0.5

0.995 if upgrade to Pro: <https://fpjs.dev/pro>

User agent:

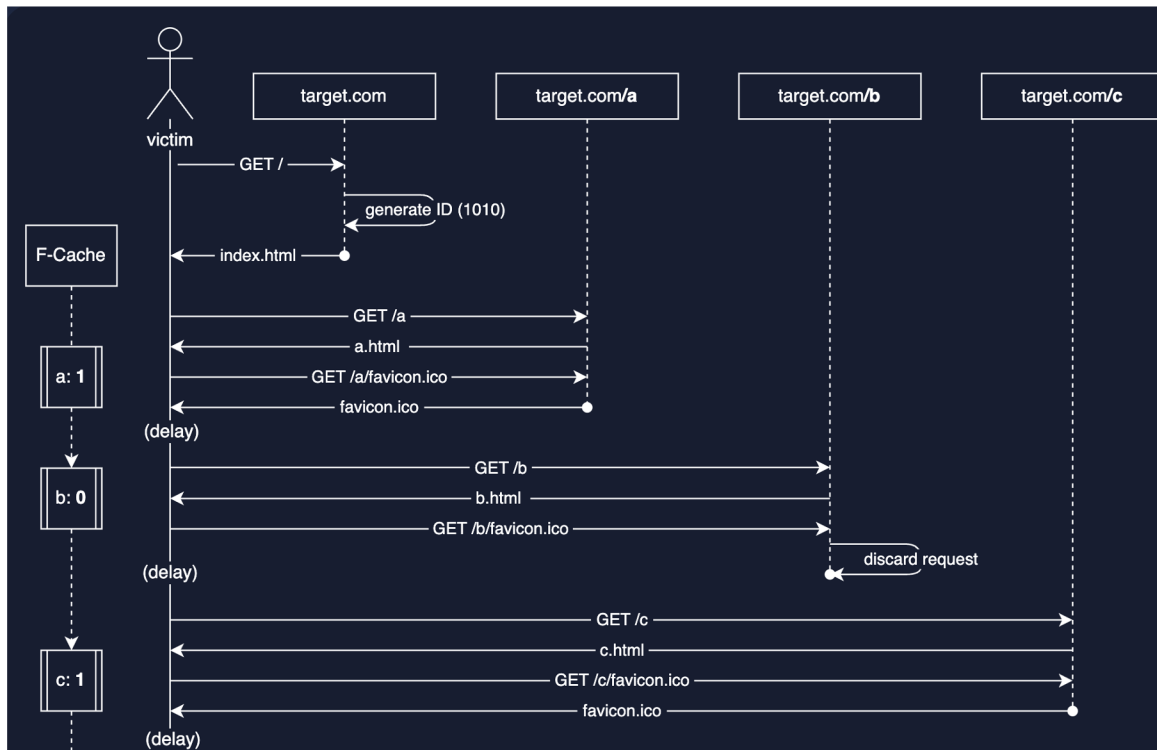
Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36

Entropy components:

```
{
  "fonts": {
    "value": [
      "Arial Unicode MS",
      "Gill Sans",
      "Helvetica Neue",
      "Menlo"
    ],
    "duration": 78
  },
  "domBlockers": {
    "duration": 18
  }
}
```

SuperCookie

<https://supercookie.me/>



Visitor

Server

last_id := 12

see
Read

$N = \text{floor}(\log_2(\text{id})) + 1$
...
get_N_routes(N)

Generate
Vector

/a

/b

/c

/d

N = 4

redirect N-times

Browser

Read
ID

F-Cache

(/a) 0

(/b) 1

(/c) 1

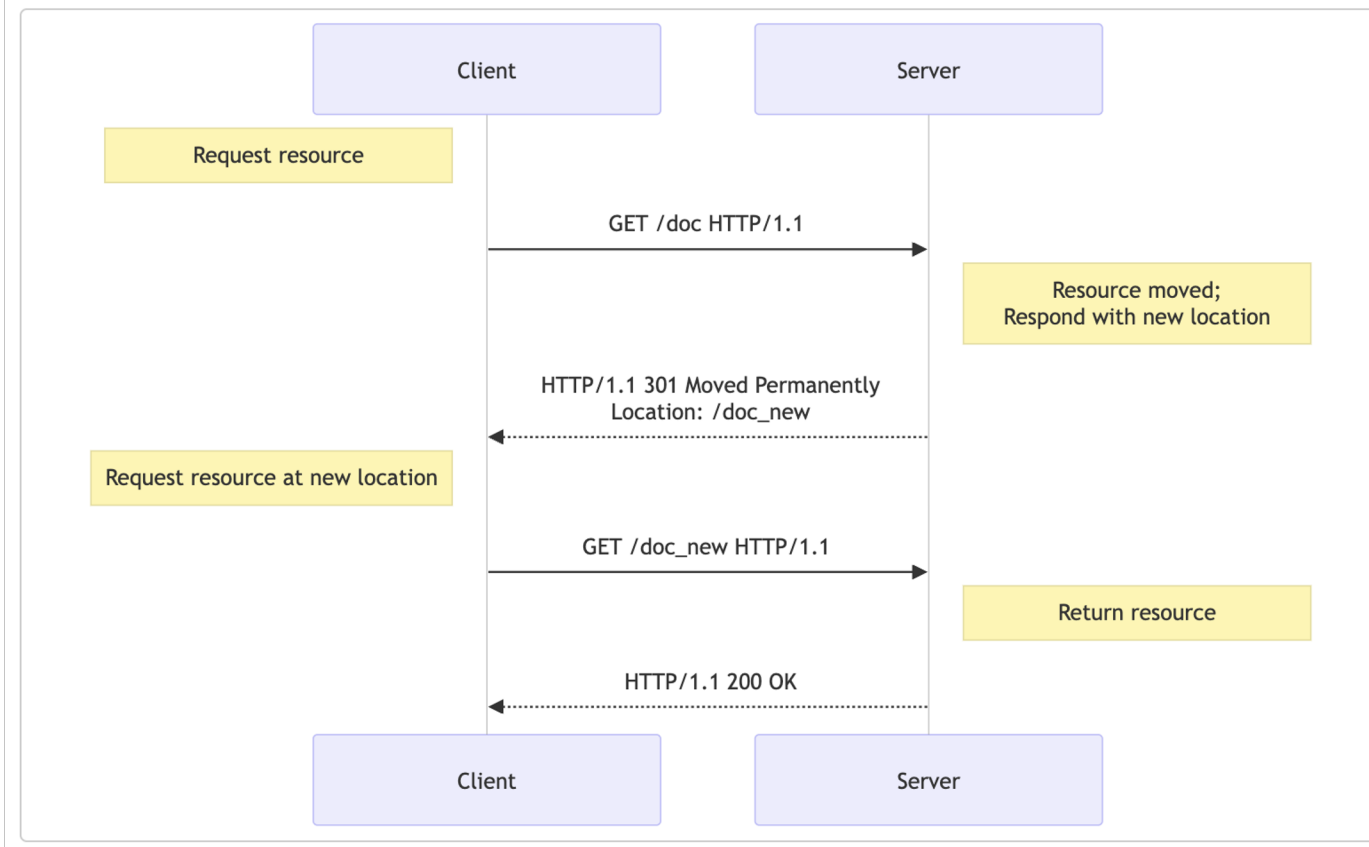
(/d) 0

binary_to_dez(0110...0)

Read "6" done

Редиректы

Редиректы



Редиректим

- Meta редирект

```
<head>
```

```
<meta http-equiv="refresh" content="0; URL=http://www.example.com/" />
```

```
</head>
```

- JavaScript редирект

```
window.location = "http://www.example.com/";
```

- HTTP редирект

```
echo -e "HTTP/1.1 301 OK\nLocation: https://codex.so\n\n" | nc -l 9001
```

<https://curl.se/docs/manpage.html>

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Location>

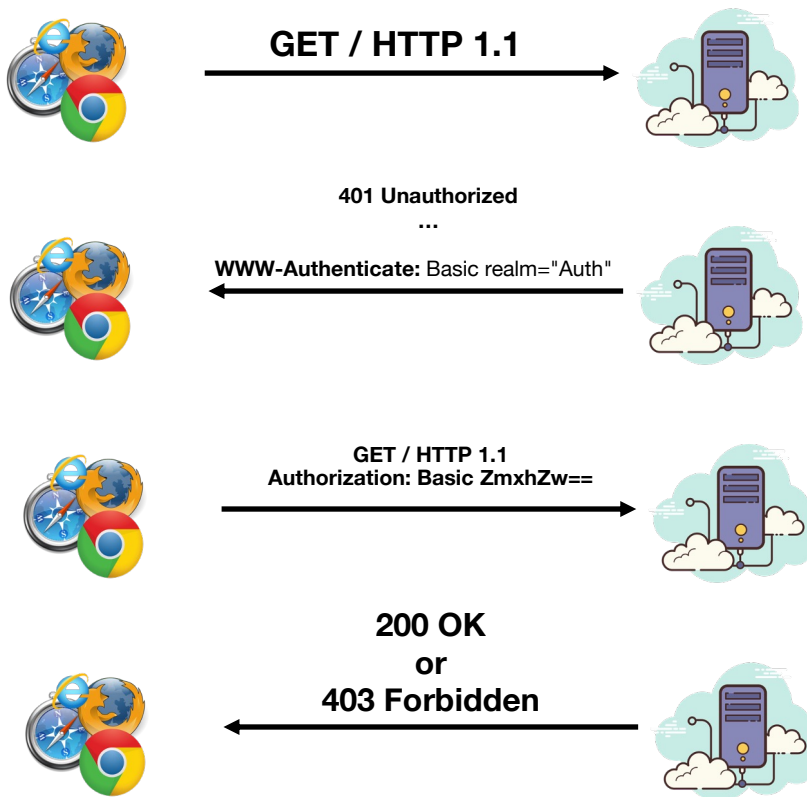
Коды редиректа

Заголовок ответа **Location** указывает **URL**, на который следует перенаправить страницу. Он имеет значение только в том случае, если обслуживается с кодом 3XX (redirection) или 201 (created).

- 1) 303 See Other — (PUT, POST), перенаправление производится не на новый ресурс, а на другую страницу, например, страницу подтверждения или страницу с результатами загрузки. Метод, используемый для отображения страницы, на которую производится перенаправление – **всегда GET**.
- 2) **Permanent** Redirect попадают в кэш
 - а) 301 Moved Permanently – используются для GET, HEAD (может поменять **method** на GET)
 - б) 308 Permanent Redirect – не меняет **method** и **body**
- 3) **Temporary** Redirect не попадают в кэш
 - а) 302 Found – исторически не должен был менять **method**, но сейчас используют для GET, HEAD
 - б) 307 Temporary Redirect – не меняет **method** и **body**

Основы аутентификации

Basic Authentication



Вход

http://localhost:5002

Имя пользователя

Пароль

Отмена

Войти

1. Сервер запрашивает

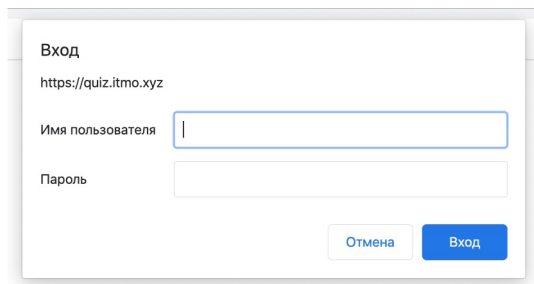
```
echo -e "HTTP/1.1 401 UNAUTHORIZED\nWWW-Authenticate: Basic  
realm=\"Auth\"\n\n" | nc -l 9003
```

Код 401 UNAUTHORIZED

Заголовок WWW-Authenticate

<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/WWW-Authenticate>
https://developer.mozilla.org/en-US/docs/Web/HTTP/Authentication#basic_authentication_scheme

2. Клиент отвечает

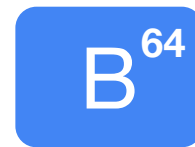


Вход
https://quiz.itmo.xyz

Имя пользователя

Пароль

username:password



dXNlcm5hbWU6cGFzc3dvcmQ=

```
echo -e "GET / HTTP/1.1\nAuthorization: Basic dXNlcjoxMjM=\n\n" | nc localhost 5002
```

AJAX

AJAX

```
$('#a#calculate').bind('click', function () {  
    $.getJSON('/_add_numbers', {  
        a: $('#input[name="a"]').val(),  
        b: $('#input[name="b"]').val()  
    }, function (data) {  
        $("#result").text(data.result);  
    });  
    return false;  
});
```

```
const response = await fetch('/_add_numbers?' +  
    new URLSearchParams({  
        a: $('#input[name="a"]').val(),  
        b: $('#input[name="b"]').val()  
    }));  
const data = await response.json();  
$("#result").text(data.result);  
return false;
```

HTMX

```
<script src="https://unpkg.com/htmx.org@1.9.10"></script>
```

```
<button hx-post="/clicked" hx-swap="outerHTML">
```

Click Me

```
</button>
```

✕	Headers	Preview	Response	Initiator	Timing	Cookies
▼ General						
Request URL:		http://localhost:5002/clicked				
Request Method:		POST				
Status Code:		● 200 OK				
Remote Address:		127.0.0.1:5002				
Referrer Policy:		strict-origin-when-cross-origin				

API и REST

Что такое API

API (Application Programming Interface) в контексте Веб – интерфейс взаимодействия приложений через Веб и в Веб.

- SOAP API (Simple Object Access Protocol) – основанный на XML протокол. Простой, не очень гибкий, был более популярен в прошлом.
- RPC API – удаленный вызов выполнения функций сервера с получением результатов выполнения.
- **REST API** – самый популярный сейчас способ работы с API с транспортом на основе HTTP (* websocket).
- *GraphQL API – язык запросов к API и среда выполнения для выполнения этих запросов с учетом графовой структуры данных.*

REST API

Six Guiding Principles of REST

01

Uniform Interface

02

Client-Server

03

Stateless

04

Cacheable

05

Layered System

06

Code on Demand (Optional)

Основные сущности

Ресурс — основная сущность(документ, изображение, любые данные).

Идентификатор ресурса – уникальный ID

```
{
  "id": 123,
  "title": "What is REST",
  "content": "REST is an architectural style for building web
services...",
  "published_at": "2023-11-04T14:30:00Z",
  "author": {
    "id": 456,
    "name": "John Doe",
    "profile_url": "https://example.com/authors/456"
  },
}
```


Пример использования API

- Сервис выдает погоду по локации
 - <https://openweathermap.org/api>
1. Читаем документацию — <https://openweathermap.org/api/one-call-3>
 2. Первым делом разбираемся с авторизацией
 3. Регистрируемся и получаем API ключ, заполняем заявку

Бесплатный сервис без ключа

<https://open-meteo.com/>

How to make an API call

API call

```
https://api.openweathermap.org/data/3.0/onecall?lat={lat}&lon={lon}&exclude={part}&appid={API key}
```



Weather API пример

```
curl "https://api.open-  
meteo.com/v1/forecast?longitude=30.314997&latitude=59.938784&  
current=temperature_2m,wind_speed_10m"
```

<https://open-meteo.com/en/docs/geocoding-api/#name=Moscow>

The Cat API пример

`https://api.thecatapi.com/v1/images/search?limit=10&breed_ids=beng&api_key=<...>`

```
import requests
from pathlib import Path

results = requests.get("https://api.thecatapi.com/v1/images/search", {
    "limit": 20,
    "breed_ids": "beng",
    "api_key": "<API_KEY>"
}).json()

for result in results:
    with (Path("cats") / f"{result['id']}.{result['url'].split('.')[-1]}" ).open('wb') as w:
        w.write(requests.get(result['url']).content)
    print(f"Saved cat {result['id']}")
```

<https://thecatapi.com/>

Swagger

Swagger — это инструмент, который помогает разработчикам создавать, документировать и проверять API.

- автоматически создавать документацию для API на основе спецификации
- автоматически генерировать клиентский код для работы с API
- визуализация функциональности API

<https://petstore.swagger.io/#>

Как генерируются спецификации

- С помощью визуального редактора <https://editor.swagger.io/>
- Автоматически из комментариев функций веб-фреймворка
- При помощи автоматических утилит, анализирующих код ([Swagger JSDoc](#))

The screenshot displays the Swagger Editor interface. On the left, the OpenAPI 3.0 specification is written in YAML format. The specification includes an API title, description, and a list of endpoints. On the right, the visual representation of the API is shown, listing the endpoints with their HTTP methods and descriptions.

```
1 openapi: 3.0.3
2 info:
3   title: Swagger Petstore - OpenAPI 3.0
4   description: |-
5     This is a sample Pet Store Server based on the OpenAPI 3.0 specification.
6     You can find out more about
7     Swagger at [https://swagger.io](https://swagger.io). In the third iteration
8     of the pet store, we've switched to the design first approach!
9     You can now help us improve the API whether it's by making changes to the
10    definition itself or to the code.
11    That way, with time, we can improve the API in general, and expose some of
12    the new features in OAS3.
13
14    _If you're looking for the Swagger 2.0/OAS 2.0 version of Petstore, then
15    click [here](https://editor.swagger.io?url=https://petstore.swagger.io
16    /v2/swagger.yaml). Alternatively, you can load via the `Edit > Load
17    Petstore OAS 2.0` menu option!_
18
19    Some useful links:
20    - [The Pet Store repository](https://github.com/swagger-api/swagger-petstore
21    )
22    - [The source API definition for the Pet Store](https://github.com/swagger
23    -api/swagger-petstore/blob/master/src/main/resources/openapi.yaml)
24
25 termsOfService: http://swagger.io/terms/
26 contact:
27   email: apiteam@swagger.io
28 license:
29   name: Apache 2.0
30   url: http://www.apache.org/licenses/LICENSE-2.0.html
```

The visual representation on the right shows the following endpoints:

- PUT** `/pet` Update an existing pet
- POST** `/pet` Add a new pet to the store
- GET** `/pet/findByStatus` Finds Pets by status
- GET** `/pet/findByTags` Finds Pets by tags
- GET** `/pet/{petId}` Find pet by ID
- POST** `/pet/{petId}` Updates a pet in the store with form data
- DELETE** `/pet/{petId}` Deletes a pet
- POST** `/pet/{petId}/uploadImage` uploads an image

Форматы данных

Требования к текстовым форматам (не бинарным)

- Передача данных по HTTP, Websocket, ...
- Хранение данных в БД
- Синтаксис конфигурационных файлов
- Сохранение логов, метрик, ...
- Совместимость формата хранения данных и синтаксиса языка программирования
- Читаемость человеком
- Легковесность, минималистичность, экономия трафика

Сериализация и Десериализация

Сериализация — процесс перевода структуры данных в бинарную последовательность. Обратной к операции сериализации является операция десериализации (структуризации) — создание структуры данных из битовой последовательности.



XML

XML зародился как язык разметки. Предназначался для хранения, передачи и восстановления произвольных данных».

Это самый популярный формат, используемый даже сегодня из-за разнообразия использования и богатых определений схемы. Популярность также возросла благодаря встроенной поддержке в различных языках программирования, библиотеках и продуктах.

```
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this
weekend!</body>
</note>
```

XSLT

```
<?xml version="1.0" encoding="UTF-8"?>
<html xsl:version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
<body style="font-family:Arial;font-size:12pt;background-color:#EEEEEE">
<xsl:for-each select="breakfast_menu/food">
  <div style="background-color:teal;color:white;padding:4px">
    <span style="font-weight:bold"><xsl:value-of select="name"/> - </span>
    <xsl:value-of select="price"/>
  </div>
  <div style="margin-left:20px;margin-bottom:1em;font-size:10pt">
    <p>
      <xsl:value-of select="description"/>
      <span style="font-style:italic"> (<xsl:value-of select="calories"/> calories
per serving)</span>
    </p>
  </div>
</xsl:for-each>
</body>
</html>
```

JSON

Формат нотации объектов JavaScript появился как синтаксис сериализации данных между веб-приложениями и серверами.

В основе простая идея двух структур:

- пары *имя + значение*. В различных языках это реализуется как **объект, запись, структура, словарь, хеш-таблица, список ключей или ассоциативный массив**.
- Упорядоченный список значений. В большинстве языков это реализуется как **массив, вектор, список или последовательность**.

<https://www.json.org/json-en.html>

YAML

Человекочитаемый язык сериализации данных. Он обычно используется для файлов конфигурации и в приложениях, где данные хранятся или передаются.

<https://yaml.org/> <https://yaml.org/spec/1.2.2/>

YAML ориентирован на те же цели, что и XML, но имеет минималистичный синтаксис.

```
#Comment: This is a supermarket list using
YAML
#Note that - character represents the list
---
food:
  - vegetables: tomatoes #first list item
  - fruits: #second list item
    citrics: oranges
    tropical: bananas
    nuts: peanuts
```

```
# ASCII Art
--- |
  \//||\//||
  // ||  ||__
```

Использование формата

JSON

```
JSON.stringify({"firstName":"John", "lastName":"Doe"})  
JSON.parse('{"firstName":"John","lastName":"Doe"}')
```

XML

```
parser = new DOMParser();  
xmlDoc = parser.parseFromString("<bookstore><book>" +  
" <title>Everyday Italian</title>" +  
" <author>Giada De Laurentiis</author>" +  
" <year>2005</year>" +  
" </book></bookstore>", "text/xml");  
const s = new XMLSerializer();  
const str = s.serializeToString(xmlDoc);
```

YAML

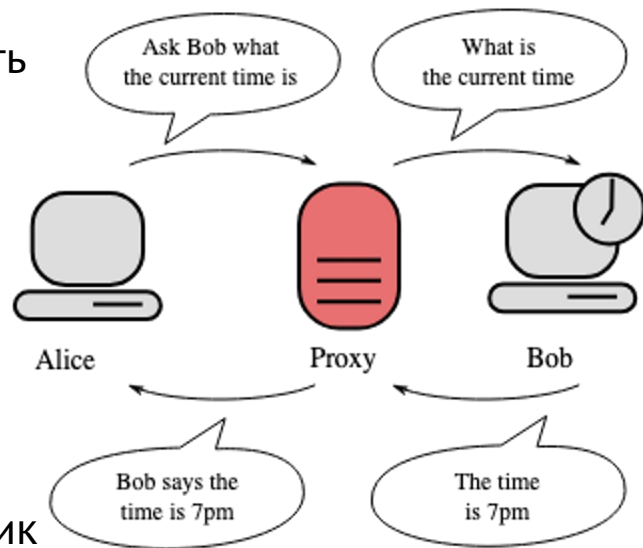
<https://nodeca.github.io/js-yaml/>

Прокси

Что такое прокси?

Прокси сервер — программа, выступающая в роли сетевого посредника, позволяющая клиентам выполнять косвенные запросы к другим сетевым службам

- HTTP(S) прокси — перенаправляет только HTTP трафик (для HTTPS выполняется MiTM)
<https://mitmproxy.org/>
 - Прозрачным прокси называется тот, в котором трафик перенаправляется неявно (подменяются заголовки)
- Socks5 прокси — перенаправляет TCP и UDP трафик
- Кастомные протоколы и утилиты (TCP/UDP/HTTP/...)
 - <https://github.com/ginuerzh/gost>
 - <https://shadowsocks.org/guide/what-is-shadowsocks.html>



Плюсы и минусы прокси

HTTP:

- Легкая имплементация анонимности
- Поддержка всеми браузерами
- Проксирование DNS запросов
- Работает только с HTTP :(Нет доверия стороннему прокси-серверу

Socks:

- Поддержка любых протоколов (например, SSH)
- Сложности в некоторых браузерах (например, некоторые версии Firefox выполняют DNS запросы от имени клиента)

Пример выполнения

```
docker run --rm -p 127.0.0.1:3129:3128 mjemala/squid
```

```
curl -x http://localhost:3129 -U proxy:proxy http://ifconfig.me/all.json -vvv
```

```
→ ~ curl -x http://localhost:3129 -U proxy:proxy http://ifconfig.me/all.json -vvv
* Trying [::1]:3129...
* connect to ::1 port 3129 failed: Connection refused
* Trying 127.0.0.1:3129...
* Connected to localhost (127.0.0.1) port 3129
* Proxy auth using Basic with user 'proxy'
> GET http://ifconfig.me/all.json HTTP/1.1
> Host: ifconfig.me
> Proxy-Authorization: Basic cHJveHk6cHJveHk=
> User-Agent: curl/8.4.0
> Accept: */*
> Proxy-Connection: Keep-Alive
>
< HTTP/1.1 200 OK
< Server: fasthttp
< Date: Mon, 04 Mar 2024 16:17:51 GMT
< Content-Type: application/json
< Content-Length: 228
< access-control-allow-origin: *
< X-Cache: MISS from a3e0392ec981
< X-Cache-Lookup: MISS from a3e0392ec981:3128
< Via: 1.1 google, 1.1 a3e0392ec981 (squid/3.3.8)
< Connection: keep-alive
<
* Connection #0 to host localhost left intact
{"ip_addr":"77.234.199.23","remote_host":"unavailable","user_agent":"curl/8.4.0","port":"43654","method":
":GET","mime":"*/*","via":"1.1 a3e0392ec981 (squid/3.3.8), 1.1 google","forwarded":"172.17.0.1,77.234.
199.23,34.117.118.44"}

```

Заголовки Proxy

- Proxy-Authorization
 - Заголовок для авторизации на Proxy сервере
- Via
 - Добавляется прокси-серверами, и может присутствовать в заголовках запроса или ответа. Он используется для отслеживания пересылки сообщений, предотвращения заикливания запросов и т.п.
 - *Via 1.1 Google*
- X-Forwarded-For
 - Заголовок для определения IP-адреса клиента, подключающегося к веб-серверу через прокси-сервер. Его добавляет прокси сервер (если того желает)
 - Сохраняется цепочка из всех IP адресов: 172.17.0.1,77.234.199.23,34.117.118.44

Как шел запрос

1. От клиента к прокси

```
> Transmission Control Protocol, Src Port: 54897, Dst Port: 3129,  
v Hypertext Transfer Protocol  
  > GET http://ifconfig.me/all.json HTTP/1.1\r\n  
    Host: ifconfig.me\r\n  
  > Proxy-Authorization: Basic cHJveHk6cHJveHk=\r\n  
    User-Agent: curl/8.4.0\r\n  
    Accept: */*\r\n  
    Proxy-Connection: Keep-Alive\r\n  
    \r\n
```

4. От прокси к клиенту

```
> Transmission Control Protocol, Src Port: 3129, Dst Port: 54897  
> [2 Reassembled TCP Segments (538 bytes): #11(310), #12(228)]  
v Hypertext Transfer Protocol  
  > HTTP/1.1 200 OK\r\n  
    Server: fasthttp\r\n  
    Date: Mon, 04 Mar 2024 16:49:50 GMT\r\n  
    Content-Type: application/json\r\n  
  > Content-Length: 228\r\n  
    access-control-allow-origin: *\r\n  
    X-Cache: MISS from a3e0392ec981\r\n  
    X-Cache-Lookup: MISS from a3e0392ec981:3128\r\n  
    Via: 1.1 google, 1.1 a3e0392ec981 (squid/3.3.8)\r\n  
    Connection: keep-alive\r\n  
    \r\n
```

2. От прокси к цели

```
v Hypertext Transfer Protocol  
  > GET /all.json HTTP/1.1\r\n  
    Host: ifconfig.me\r\n  
    User-Agent: curl/8.4.0\r\n  
    Accept: */*\r\n  
    Via: 1.1 a3e0392ec981 (squid/3.3.8)\r\n  
    X-Forwarded-For: 172.17.0.1\r\n  
    Cache-Control: max-age=259200\r\n  
    Connection: keep-alive\r\n  
    \r\n
```

3. От цели к прокси

```
v Hypertext Transfer Protocol  
  > HTTP/1.1 200 OK\r\n  
    server: fasthttp\r\n  
    date: Mon, 04 Mar 2024 16:43:57 GMT\r\n  
    content-type: application/json\r\n  
  > Content-Length: 228\r\n  
    access-control-allow-origin: *\r\n  
    via: 1.1 google\r\n  
    \r\n
```

Списки прокси

<https://spys.one/en/socks-proxy-list/>

<https://hidemy.io/en/proxy-list/> и любые другие

```
curl -x socks5h://192.111.129.145:16894 -k http://ifconfig.me/all.json -vvv
```

- -x – URL до прокси
- -k – игнорить SSL
- -U – авторизация

IP address	Port	Country, City	Speed	Type	Anonymity	Latest update
23.227.38.216	80	Canada	160 ms	HTTP	no	1 minutes
188.114.96.171	80	Netherlands Amsterdam	260 ms	HTTP	no	1 minutes
141.101.120.110	80		220 ms	HTTP	no	1 minutes
173.245.49.180	80	France Paris	900 ms	HTTP	no	1 minutes
172.67.182.19	80	United States	220 ms	HTTP	no	1 minutes
172.67.255.13	80	United States	280 ms	HTTP	no	1 minutes

Полезные ссылки

- <https://restfulapi.net/>

Вопросы?